



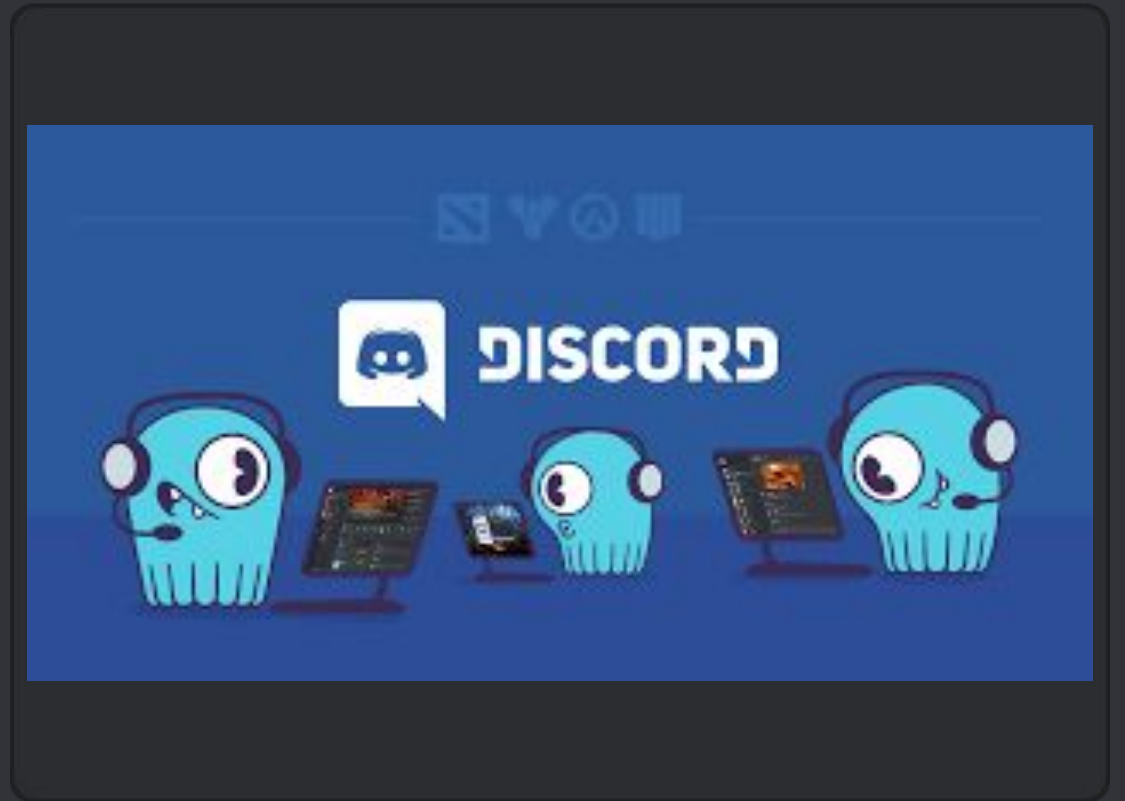
Escalando para **Trilhões** de Mensagens

Estudo de Caso: A Migração do Discord para o ScyllaDB

Baseado no case técnico de otimização estrutural e algoritmos

O Problema de Escala

- Em 2017, o Discord adotou o Apache Cassandra, chegando ao auge de 177 nós em 2021 com 140 milhões de usuários mensais.
- Ao atingir trilhões de mensagens armazenadas, a arquitetura começou a entrar em colapso devido à complexidade da busca.
- O banco sofreu falhas severas de paginação e a latência de carregamento disparou para os usuários.
- As frequentes manutenções tornaram a arquitetura antiga um dreno financeiro inviável para a empresa.



Por que o **Cassandra** falhou?



Garbage Collector

Escrito na linguagem Java, o Cassandra sofria com o "Garbage Collector" preguiçoso, que gerava travamentos repentinos para limpar a memória RAM excedente.



Queda de Desempenho

A arquitetura não conseguia gerir as buscas lineares no disco, tornando o carregamento do histórico do chat inaceitavelmente lento e custoso.



Custo Exponencial

Para compensar a ineficiência algorítmica, a equipe era forçada a escalar os clusters horizontalmente, aumentando brutalmente os gastos com a nuvem.

A Decisão Técnica: ScyllaDB

Um grande passo que o Discord tomou, mas também, muito arriscado!

O Poder da Linguagem C++

Ao contrário do Cassandra, o ScyllaDB é construído nativamente em C++. Isso elimina a latência induzida pela máquina virtual Java e garante um controle manual e direto da alocação de memória no servidor Linux, evitando travamentos em horário de pico.

Arquitetura Shard-per-Core

Utilizando o poderoso framework Seastar, o ScyllaDB vincula frações do banco de dados a núcleos individuais do processador físico (Shard-per-Core). Isso potencializa as operações de entrada e saída (I/O) em ambientes hiper-escaláveis.

Estratégia de Migração

Abordagem Progressiva

Para evitar tempo de inatividade em uma base com trilhões de registros, o Discord arquitetou uma transição de dupla leitura.

Eles implementaram "Data Services" que permitiram ao Cassandra e ao ScyllaDB coexistirem, injetando os mesmos dados simultaneamente para validar a consistência computacional.



Impacto na Infraestrutura

77

Nós no ScyllaDB (Anterior: 177)

Otimização Extrema

Ao otimizar as estruturas de dados fundamentais do sistema, o Discord desativou 100 servidores de nuvem. Eles alcançaram um processamento exponencialmente maior enquanto utilizavam menos da metade do hardware físico anterior.

Comparativo de Latência

Apache Cassandra

145 ms

ScyllaDB

15
ms

O pico de latência, que sofria fortes flutuações e travava o carregamento do aplicativo, foi brutalmente estabilizado. A busca linear no disco cedeu lugar a um tempo de resposta algorítmico super otimizado.

Análise Crítica & Proposta Alternativa

O Custo da Alta Performance

- Apesar do enorme sucesso técnico, o modelo do ScyllaDB exige servidores premium otimizados com discos SSD NVMe, cobrando seu preço.
- A calculadora de custos revela que os clusters operam com valores altíssimos, partindo da casa de \$18.000 dólares mensais.
- Este é um trade-off clássico na computação corporativa: trocar orçamento por tempo de execução otimizado (Time-Space Trade-off).

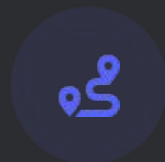


Proposta **Alternativa** do Grupo



Data Tiering

Transferir mensagens com mais de um ano inativas (Cold Data) para serviços de arquivamento (Object Storage padrão), desafogando o hardware NVMe principal.



Sink Routing

Implementar um roteador de busca que verifica a data antes da consulta, direcionando requisições novas ao ScyllaDB e as muito antigas ao armazenamento barato.



Decisão Híbrida

Essa arquitetura reduziria as contas milionárias de nuvem. Aceitamos um pequeno atraso de carregamento em 1% das buscas mais antigas, otimizando os 99% restantes.

P, NP ou NP-completo:

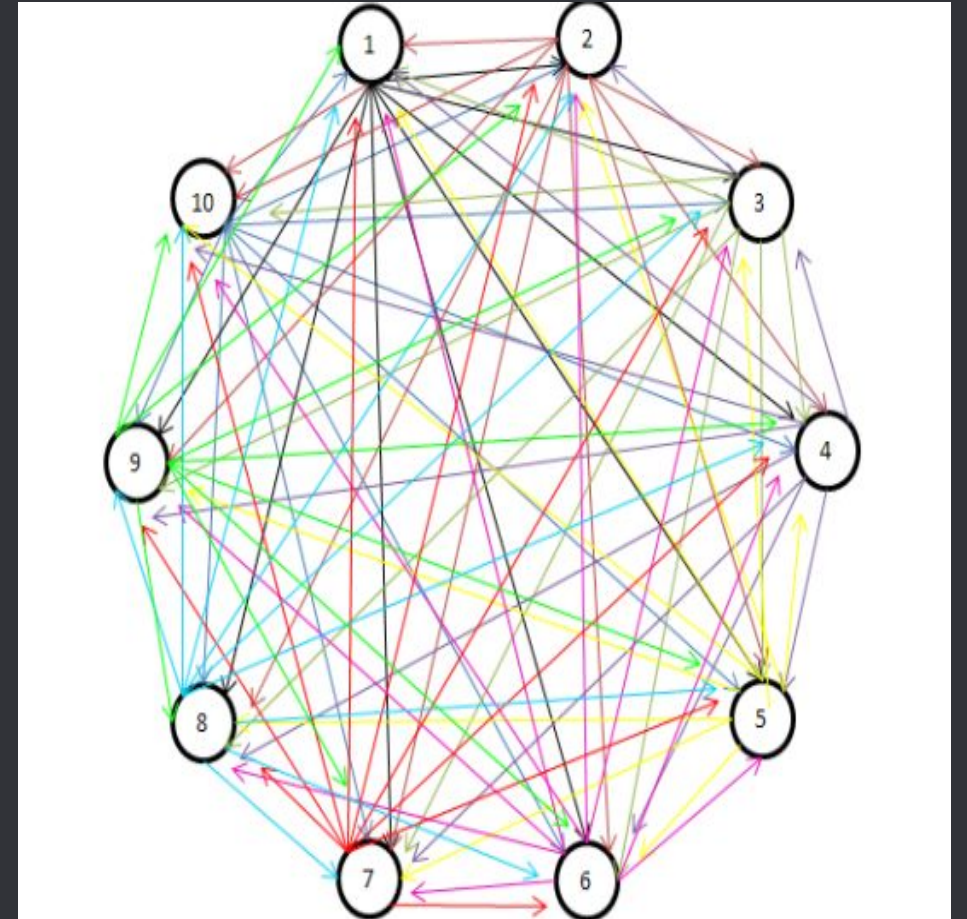
O problema se enquadra firmemente na classe P (Tempo Polinomial). As operações fundamentais do Discord são a inserção e a busca de mensagens, que teoricamente devem ser resolvidas em tempo constante $O(1)$ ou logarítmico $O(\log n)$ através de índice.

A complexidade do caso não advém de uma explosão combinatória ou de um problema intratável (como seria em problemas NP-Completo), mas sim do gargalo de infraestrutura (I/O) e da volumetria colossal de dados.

Mas por que não é um problema NP?

Encontrar a rota mais curta passando por todos servidores seria um problema NP (Caixeiro Viajante). Mas simplesmente ir até um servidor e ler uma mensagem é um problema P.

A dificuldade do Discord não era a complexidade teórica do algoritmo, era a física do disco rígido não dar conta de ler tantos dados ao mesmo tempo."



Créditos

Discord: Trilhões de mensagens com ScyllaDB

por: **Master Dev**

